# Вопросы жюри и ответы

Заготовка для защиты. Часть I — продуктовые вопросы, часть II — технические. Ответы короткие, со ссылками на файлы репозитория; места, где решение ещё не принято, помечены явно.

## Часть I. Продуктовые вопросы

### 1. Какую проблему решает система?

Она объединяет в одном рабочем процессе взаимодействия с вузами, статусы, задачи, комментарии, документы и управленческие отчёты.

### 2. Кто основной пользователь?

KAM, который ведёт портфель вузов. Руководитель использует отчёты, а администратор управляет справочниками и workflow.

### 3. Почему Kanban?

Kanban быстро показывает узкие места: количество карточек на каждом этапе, просроченные действия и следующий шаг по конкретному вузу.

### 4. Можно ли изменить 14 этапов?

Да. Администратор может добавить, переименовать, отсортировать, включить, выключить или удалить этап с переносом взаимодействий.

### 5. Что произойдёт при отключении этапа с карточками?

Backend отклонит опасную операцию. Сначала нужно перенести активные взаимодействия либо удалить этап с явно выбранным целевым этапом.

### 6. Как ограничивается доступ?

JWT-аутентификация и RBAC проверяются на backend. Административные операции недоступны роли без соответствующего разрешения.

### 7. Где хранятся данные?

Сейчас поддерживаются два контура. Основной — Yandex Cloud: BM, nginx, PostgreSQL и Object Storage, данные размещаются в РФ. Альтернативный — Render (API) + Vercel (frontend) + Supabase (PostgreSQL), используется как публичный демонстрационный стенд. Код один и тот же, различия — в переменных окружения.

### 8. Почему основным контуром выбран Yandex Cloud?

Требования 152-ФЗ к локализации баз данных граждан РФ и приказ ФСТЭК № 117 предполагают размещение в российском контуре. Yandex Cloud даёт аттестованную инфраструктуру, а весь стек уже контейнеризован, поэтому перенос выполняется сменой переменных окружения без правки кода.

### 9. Как обеспечивается 152-ФЗ?

В продукте есть аудит, RBAC, HTTPS/CORS, политика приватности и cookie consent. Локализация данных требует отдельного подтверждения выбранного production-контура.

### 10. Какие данные считаются персональными?

Контакты пользователей и представителей вузов, учетные данные и технические идентификаторы. Их нельзя помещать в тестовые данные и публичные скриншоты.

### 11. Как работает экспорт?

Frontend выполняет авторизованный Axios-запрос как Blob, backend возвращает Content-Disposition, после чего браузер сохраняет XLSX, XLS или PDF.

### 12. Как проверяется производительность?

В репозитории есть Locust-сценарий: логин, отчёты, Kanban и справочники. Целевые критерии — 50 пользователей, 60 секунд, p95 менее 1000 мс.

### 13. Есть ли уже фактические результаты нагрузки?

Да, локальный прогон выполнен: 50 пользователей / 60 секунд, 1613 запросов, 0 ошибок, aggregate p95 93 мс; у `/api/reports` p95 7 мс. Отчёты лежат в `tests/load/report.html` и `tests/load/report-10-reports.html`. Это цифры локального стенда, а не подтверждённый SLA на BM заказчика — его нужно перепроверить на целевом контуре.

### 14. Что будет при недоступности backend?

UI показывает загрузку или ошибку, не маскируя отсутствие данных. После восстановления соединения TanStack Query повторяет запрос.

### 15. Как система масштабируется?

Frontend статичен, backend можно запускать несколькими экземплярами, а PostgreSQL масштабируется средствами выбранного managed-контура.

### 16. Почему FastAPI?

Он даёт типизированные схемы, асинхронный стек и автоматическую OpenAPI- документацию при небольшом объёме инфраструктурного кода.

### 17. Как предотвращается потеря истории?

Изменяющие операции проходят audit middleware, а workflow-операции сохраняют связь карточки с этапом и пользователем.

### 18. Как восстанавливаются данные?

Для Yandex Cloud подготовлен ежедневный `pg_dump` с хранением семи последних копий. Восстановление должно регулярно проверяться на тестовой базе.

### 19. Что сделано для мобильных устройств?

Метрики перестраиваются в сетку, формы становятся вертикальными, таблицы получают локальную горизонтальную прокрутку, а footer учитывает safe-area. Cookie-баннер на телефоне ограничен 55vh, кнопки встают столбиком, модалка настроек укладывается в экран.

### 20. Какой следующий шаг после защиты?

Провести реальный нагрузочный тест, закрыть browser smoke test, подтвердить контур хранения данных и согласовать миграцию production в российское облако.

## Часть II. Технические вопросы

### 1. Как работает двухсторонняя интеграция с LMS и CMS?

Контракт обмена описан эндпоинтом `GET /api/integration/schema`, приём пакета — `POST /api/integration/import`, сухой прогон без записи — `POST /api/integration/preview`, выгрузка наружу — `GET /api/integration/outbound` (см. `backend/api/integration.py`). Повторный импорт сопоставляет карточку по номеру договора и обновляет её вместо создания дубля; неизвестные поля не ломают разбор, а возвращаются в `warnings`. Забор из внешней системы (`GET /api/integration/pull/{source}`) пока читает заглушку, потому что контракт LMS/CMS заказчиком не предоставлен — переход на живой источник сводится к замене источника данных.

### 2. Почему не версионируете workflow?

Версионирование не требовалось ТЗ, а изменение этапа действует на все активные карточки сразу — это осознанный выбор в пользу простоты поддержки администратором. Этапы разделены полем `scope` на два набора: B2B (14 этапов ИТ-Школы) и B2C (упрощённая воронка), см. `backend/models/entities.py`. Историю изменений частично закрывает аудит: переход фиксируется записью в `action_logs` со старым и новым значением. Если версии понадобятся, следующий шаг — отдельная таблица версий workflow поверх текущей схемы.

### 3. Как реализован аудит-лог?

Записи создаёт middleware `backend/middleware/audit.py` на каждый успешный изменяющий запрос (`POST`, `PUT`, `PATCH`, `DELETE` со статусом меньше 400); `GET/OPTIONS/HEAD` и эндпоинты логина и health пропускаются. В `action_logs` пишутся пользователь, действие (CREATE/UPDATE/DELETE), тип и id сущности, старое и новое значение и IP-адрес. Читается журнал администратором через `GET /api/audit` с фильтрами и выгрузкой в CSV (`backend/api/audit.py`) — данные можно не только накапливать, но и показать проверяющему.

### 4. Что с 152-ФЗ, где хранятся персональные данные?

Персональными считаются контакты представителей вузов, учётные данные пользователей и технические идентификаторы; они не попадают в тестовые данные и публичные скриншоты. Основной контур — Yandex Cloud: BM, nginx, PostgreSQL и Object Storage в РФ, что закрывает локализацию данных граждан РФ (`docs/architecture/deployment-yandex-cloud.md`). Технические меры — RBAC, аудит, HTTPS/CORS и заголовки безопасности (`backend/middleware/security_headers.py`). Локализация конкретного production-контура требует отдельного подтверждения заказчиком, это вынесено в план следующего шага.

### 5. Как работает RBAC на уровне запросов?

Роль берётся из JWT, а уровень доступа проверяется зависимостями FastAPI: `get_current_user`, `require_manager_or_admin`, `require_admin`. Точечные операции по id проходят через `backend/api/access.py`: KAM (роль `user`) видит только свои карточки, проверка идёт по `assigned_kam_id`, а чужой id даёт 403, а не открытие карточки. Логика вынесена в один модуль, чтобы одинаково применяться и во взаимодействиях, и в файлах.

## 6. Как обрабатываете 50 параллельных пользователей?

Сценарий — `tests/load/locustfile.py`: логин, отчёты, Kanban и справочники. Прогон выполнен локально, отчёты лежат в `tests/load/`: **50 пользователей / 60 секунд, 1613 запросов, 0 ошибок, aggregate p95 93 мс**. Отдельный профиль `ReportsUser` дал 322 запроса, 0 ошибок, у `/api/reports` p95 7 мс. Под эту нагрузку заложены асинхронный стек (FastAPI + SQLAlchemy async), кэш агрегатов отчётов на 30 секунд (`backend/services/report_cache.py`) и индексы на фильтруемых полях карточек. Production SLA отдельно не подтверждался — это цифры локального прогона, а не гарантия для ВМ заказчика.

## 7. Что с утечками, снимали ли профайлер?

Профайлер памяти не запускался, и объявлять его результаты мы не будем — это честная позиция. Что уже ограничивает утечки: тело запроса для аудита обрезается, состояние сессии БД живёт внутри запроса (`SessionLocal` как контекстный менеджер), глобального изменяемого состояния на запрос нет, а кэш отчётов имеет TTL. Следующий шаг — прогон `tracemalloc`/`py-spy` под нагрузкой и фиксация baseline в отчёте.

## 8. Как устроен импорт XLS/XLSX, где диалоговый маппинг?

Импорт каталогов — `backend/services/excel_import.py`: `openpyxl` для XLSX, `xlrd` для XLS, структура проверяется `jsonschema`. Диалоговый маппинг колонок есть в UI (`frontend/src/pages/DirectoryPage.tsx`): после выбора файла открывается модалка «Сопоставление полей», пользователь видит до пяти предпросмотренных строк и сопоставляет колонки, а выбранная схема сохраняется в `localStorage` по типу справочника и подставляется при следующем импорте. Проверка идёт на backend через `preview`, обязательные поля нельзя оставить несопоставленными. Ошибки возвращаются по строкам (`ValidationIssue`: строка, поле, проблема, уровень — error блокирует строку, warning нет), а отчёт о них выгружается в XLSX (`backend/services/import_report.py`).

## 9. Как тестировали write-операции?

Изменяющие сценарии покрыты интеграционными тестами, которые ходят через API в реальную БД: `test_audit_log.py`, `test_interaction_access.py`, `test_user_delete.py`, `test_workflow_scope.py`, `test_integration.py`, `test_excel_import.py`, `test_json_export.py`. Всего в наборе 200 проходящих тестов, отдельно 22 теста Keycloak. Моки не применяются: проверяются реальные пути записи, ограничения целостности и запись в аудит.

## 10. Что с масштабированием на 300+?

Backend без собственного состояния, поэтому масштабируется репликами за балансировщиком; сессии, файлы и кэш вынесены в PostgreSQL, S3 и Redis/KeyDB (`docs/architecture/ARCHITECTURE.md`, раздел «Масштабируемость»). Что уже заложено в код: пул соединений, индексы под фильтрацию доски и отчётов, кэш агрегатов на 30 секунд. При росте нагрузки следующий шаг — реплики чтения PostgreSQL и вынос отчётов на отдельный пул.

## 11. Почему KeyDB, а не Redis?

Сейчас в контуре Redis 7, а KeyDB — многопоточный форк Redis с тем же протоколом, рекомендованный заказчиком как более лёгкий (`docs/STACK.md`). Переход не требует правок кода: адрес задаётся переменной окружения `redis_url`, а `backend/services/report_cache.py` работает через обычный клиент Redis. То есть это замена эксплуатационного слоя, а не переписывание логики.

## 12. Как обрабатываете удаление статуса с активными задачами?

Удаление этапа — `DELETE /api/stages/{stage_id}` с необязательным `target_stage_id` (`backend/api/stages.py`). На уровне БД стоит `ON DELETE RESTRICT`, поэтому активный этап нельзя удалить, пока на нём есть карточки: без `target_stage_id` вернётся 409 с количеством взаимодействий, а с ним карточки переносятся на целевой этап и только потом этап удаляется. Это защищает от потери задач при чистке воронки.

## 13. Как реализован экспорт в JSON?

`GET /api/reports/json` (`backend/api/reports.py`) отдаёт воспроизводимую выгрузку: `generated_at`, `total`, описание колонок из общего конфига `config/report_columns.json`, блок `storage` (S3 или local и имя бакета), применённые фильтры и сами взаимодействия с ключами файлов. По требованию Q&A Крылова выгрузка содержит не только данные карточек, но и ключи связи, чтобы принимающая сторона сопоставила объекты и скачала вложения. Колонки и фильтры те же, что в интерфейсе и в PDF/XLS-выгрузках.

## 14. Что с LLM — как обоснуете использование?

LLM применяется только для одной необязательной функции — сводки по карточке (ФТ-6), см. `backend/services/gigachat.py`. Без ключа `GIGACHAT_CREDENTIALS` функция выключена и отдаёт 503, а интерфейс скрывает кнопку; ошибки внешнего сервиса не превращаются в 500. В промпт уходят только тексты комментариев, названия задач, вуза и продукта — персональные данные исключены, а демо-сводка без обращения к модели явно помечается отдельным значением поля «Модель».

## 15. Как деплоите — Docker, Kubernetes?

Основной способ — Docker и Docker Compose: базовый `docker-compose.yml` и прод-стек `infra/yandex-cloud/docker-compose.prod.yml` с nginx (`docs/DEPLOYMENT.md`). Kubernetes в текущем контуре не используется; он указан как путь горизонтального масштабирования, когда одной ВМ перестанет хватать. Такой выбор оправдан размером нагрузки и сроком: одна ВМ с контейнерами проще в поддержке, а переход на оркестратор не требует правок кода.

## 16. Что с целостностью данных при параллельном обновлении?

Транзакции идут через сессию SQLAlchemy, а критичные инварианты проверяются на сервере: лимит активных взаимодействий на вуз `_ensure_parallel_limit` (`backend/api/interactions.py`), совпадение `scope` карточки и этапа (409), уникальность этапов по `(scope, code)` и `(scope, order)`. Явного оптимистичного блокирования (колонки версии или `SELECT FOR UPDATE`) сейчас нет — при одновременной правке одной карточки действует «последняя запись побеждает», и это задокументированный следующий шаг. Предлагаемое решение — колонка `version` или блокировка строки при конфликтных операциях.

## 17. Как считаете ранжирование программ?

Отдельного алгоритма ранжирования программ в системе нет, и мы не будем его изображать. Отчёты дают агрегаты и фильтры по этапам, вузам, продуктам и направлениям, а сортировка выполняется по колонкам выгрузки (`config/report_columns.json`). Если заказчику нужен приоритетный список, это формулируется как отдельная задача с явными весами критериев, а не подразумевается текущей логикой.

## 18. Почему Python, а не Kotlin?

ТЗ само предлагает Python и PostgreSQL, и FastAPI закрывает задачу без выигрыша от смены языка: асинхронный ввод-вывод под нагрузку 300+, типизированные схемы и общий язык типов с Pydantic, автоматическая OpenAPI-документация (`docs/architecture/ARCHITECTURE.md`).

Kotlin потребовал бы отдельной команды и переписывания без преимуществ для задач CRM. Команда владеет Python, поэтому скорость разработки и поддержки выше.

## 19. Что сделано для мобильных устройств?

Интерфейс адаптивный: метрики перестраиваются в сетку, формы становятся вертикальными, таблицы получают локальную горизонтальную прокрутку, а footer учитывает safe-area. Cookie-баннер на телефоне ограничен 55vh со скроллом внутри, кнопки встают столбиком на всю ширину, а модалка настроек укладывается в экран и не перекрывает свой футер. Отдельного нативного приложения нет — сценарий КАМ закрывается браузером на телефоне. Если потребуется, следующий шаг — PWA-оболочка поверх текущего SPA.

## 20. Что с документацией — встроена ли в платформу?

Да, документация встроена: вкладка «Помощь» (`/help`) отдаёт Markdown из статического зеркала `frontend/public/docs/`, которое собирается скриптом `scripts/sync_public_docs.py`. Рядом лежат единый PDF (`docs/rtk-crm-documentation.pdf`), отдельные руководства пользователя и администратора и архитектурные схемы. Зеркало проверяется тестом на дрейф (`tests/test_docs_mirror.py`), поэтому «Помощь» не расходится с исходниками.